

Incidente en el Núcleo — Resolución

incidente-en-el-nucleo.hs — combinarFirewalls / intensidadRestante / pasos / rutasAServ / fold / rec

Tipos de datos

```
type Potencia = Int
type Ataque   = Int
data Red = Terminal Potencia
         | Servidor Potencia Red
         | Firewall (Ataque -> Bool) Red
         | Switch   (Ataque -> Dir)  Red Red
data Dir = Izq | Der | Forward
         deriving Show
type Ruta = [Dir]

instance Show Red where
  show (Terminal p)      = "Terminal " ++ show p
  show (Servidor p r)    = "Servidor " ++ show p ++ " (" ++ show r ++ ")"
  show (Firewall _ r)    = "Firewall  (" ++ show r ++ ")"
  show (Switch _ r1 r2) = "Switch   (" ++ show r1 ++ ") (" ++ show r2 ++ ")"
```

Auxiliar: addFst

```
addFst :: (Int -> Int) -> (Int, b) -> (Int, b)
addFst f (n, y) = (f n, y)
```

Punto 1 — combinarFirewalls

```
combinarFirewalls :: Red -> Red
combinarFirewalls (Terminal n)      = Terminal n
combinarFirewalls (Servidor n r)    = Servidor n (combinarFirewalls r)
combinarFirewalls (Firewall f r)    = combinar f (combinarFirewalls r)
combinarFirewalls (Switch fsw ri rd) = Switch fsw (combinarFirewalls ri) (combinarFirewalls rd)

combinar :: (Ataque -> Bool) -> Red -> Red
combinar f (Firewall f2 r) = Firewall (\a -> f a && f2 a) r
combinar f r               = Firewall f r
```

intensidadRestante

```
intensidadRestante :: Ataque -> Red -> Maybe Ataque
intensidadRestante a (Terminal n)      = Just (a - n)
intensidadRestante a (Servidor n r)    = intensidadRestante (a + n) r
intensidadRestante a (Firewall f r)    = if f a
                                         then intensidadRestante a r
                                         else Nothing

intensidadRestante a (Switch fsw ri rd) =
  case fsw a of
    Der -> case (intensidadRestante a rd) of
      (Just aIzq) -> intensidadRestante aIzq ri
      Nothing -> Nothing
    _ -> case (intensidadRestante a ri) of
      (Just aDer) -> intensidadRestante aDer rd
      Nothing -> Nothing
```

pasos / pasosHastaBloqueo

```
pasosHastaBloqueo :: Ataque -> Red -> Maybe Int
pasosHastaBloqueo a r = case pasos r a of
    (p, Nothing) -> Just p
    (_, _)       -> Nothing

pasos :: Red -> Ataque -> (Int, Maybe Ataque)
pasos (Terminal n)      a = (1, (Just (a - n)))
pasos (Servidor n r)    a = addFst (+1) (pasos r (a + n))
pasos (Firewall f r)    a = if f a
                           then addFst (+1) (pasos r a)
                           else (0, Nothing)

pasos (Switch fsw ri rd) a =
    case fsw a of
        Der -> case addFst (+1) (pasos rd a) of
            (pasosDer, Nothing) -> (pasosDer, Nothing)
            (pasosDer, (Just ataqueRes)) -> addFst (+pasosDer) (pasos ri ataqueRes)
        _ -> case addFst (+1) (pasos ri a) of
            (pasosIzq, Nothing) -> (pasosIzq, Nothing)
            (pasosIzq, (Just ataqueRes)) -> addFst (+pasosIzq) (pasos rd ataqueRes)
```

rutasAServ

```
rutasAServidores :: Ataque -> Red -> [Ruta]
rutasAServidores a red = fst (rutasAServ a red)

rutasAServ :: Ataque -> Red -> ([Ruta], Maybe Ataque)
rutasAServ a (Terminal n)      = ([], Just (a - n))
rutasAServ a (Servidor n r)    = addRutasFst (\rs -> [] : map (Forward :) rs) (rutasAServ (a + n) r)
rutasAServ a (Firewall f r)    = if f a
                                   then addRutasFst (map (Forward :)) (rutasAServ a r)
                                   else ([], Nothing)

rutasAServ a (Switch fsw ri rd) =
    case fsw a of
        Der -> case addRutasFst (map (Der :)) (rutasAServ a rd) of
            (rs, Just ataqueRes) -> addRutasFst ((rs++) . (map (Izq :))) (rutasAServ a ri)
        _ -> case addRutasFst (map (Izq :)) (rutasAServ a ri) of
            (rs, Just ataqueRes) -> addRutasFst ((rs++) . (map (Der :))) (rutasAServ a rd)

addRutasFst :: ([a] -> [a]) -> ([a], b) -> ([a], b)
addRutasFst f (n, y) = (f n, y)

addServer :: Red -> [Ruta] -> [Ruta]
addServer (Servidor _ _) rs = [] : (map (Forward :) rs)
addServer _ rs = [] : rs
```

Demostración: intensidadRestante i . combinarFirewalls = intensidadRestante i

por principio de extencionalidad, sea r una red cualquiera

¿ para toda r . $(\text{intensidadRestante } i \text{ . combinarFirewalls}) r = \text{intensidadRestante } i r$?

por def. de compose, es equivalente a ver:

¿ para toda r . $\text{intensidadRestante } i (\text{combinarFirewalls } r) = \text{intensidadRestante } i r$?

por principio de inducción estructural sobre la estructura de r

```

casoBase) r = (Terminal p)
¿ intensidadRestante i (combinarFirewalls (Terminal p)) = intensidadRestante i (Terminal p) ?

casoInductivo1) r = (Servidor n r')
HI) ¡ intensidadRestante i (combinarFirewalls r') = intensidadRestante i r' !
TI) ¿ intensidadRestante i (combinarFirewalls (Servidor n r')) = intensidadRestante i (Servidor n r') ?

casoInductivo2) r = (Firewall f r')
HI) ¡ intensidadRestante i (combinarFirewalls r') = intensidadRestante i r' !
TI) ¿ intensidadRestante i (combinarFirewalls (Firewall f r')) = intensidadRestante i (Firewall f r') ?

casoInductivo3) r = (Switch f ri rd)
HI1) ¡ intensidadRestante i (combinarFirewalls ri) = intensidadRestante i ri !
HI2) ¡ intensidadRestante i (combinarFirewalls rd) = intensidadRestante i rd !
TI) ¿ intensidadRestante i (combinarFirewalls (Switch f ri rd)) = intensidadRestante i (Switch f ri rd) ?

```

Demuestro

casoBase)

```

LI)    intensidadRestante i (combinarFirewalls (Terminal p))
=
    intensidadRestante i (Terminal p)                                --combinarFirewalls.1

LD)    intensidadRestante i (Terminal p)

```

VALE ESTE CASO

casoInductivo1)

```

LI)    intensidadRestante i (combinarFirewalls (Servidor n r'))
=
    intensidadRestante i (Servidor p (combinarFirewalls r'))        --combinarFirewalls.2
=
    intensidadRestante (i + p) (combinarFirewalls r')              --intensidadRestante.2
=

LD)    intensidadRestante i (Servidor n r')
=
    intensidadRestante (i + p) r'                                    --intensidadRestante.2
=
    intensidadRestante (i + p) (combinarFirewalls r')              --HI

```

VALE ESTE CASO

casoInductivo2)

```

LI)    intensidadRestante i (combinarFirewalls (Firewall f r'))
=
    intensidadRestante i (combinar f (combinarFirewalls r))        --combinarFirewalls.3
=
    if f i then intensidadRestante i (combinarFirewalls r) else Nothing -- LEMA-COMBINAR
    -- HI
=
    if f i then intensidadRestante i r else Nothing

LD)    intensidadRestante i (Firewall f r')
=
    if f i then intensidadRestante i r' else Nothing                -- intensidadRestante.3

```

LEMA-COMBINAR

¿intensidadRestante i (combinar f red) = if f i then intensidadRestante i red else Nothing?

por casos sobre forma de red

```

red != Firewall
LI)
  intensidadRestante i (combinar f red)
=
  intensidadRestante i (Firewall f red)
=
  if f i then intensidadRestante i red else Nothing

LD)
  if f i then intensidadRestante i red else Nothing

red = Firewall
LI)
  intensidadRestante i (combinar f (Firewall f2 r))
=
  intensidadRestante i (Firewall (\a -> f a && f2 a) r)
=
  if (\a -> f a && f2 a) i then intensidadRestante a r else Nothing
=
  if f i && f2 i then intensidadRestante i r else Nothing
LD)
  if f i then intensidadRestante i (Firewall f2 r) else Nothing
=
  if f i
    then (
      if f2 i
        then intensidadRestante i r
        else Nothing
      )
    else Nothing
=
  if f i && f2 i then intensidadRestante i r else Nothing
-- propiedad del parcial

```

Punto 3 — foldRed / recRed

```

foldRed
  :: (Potencia -> b)
  -> (Potencia -> b -> b)
  -> ((Ataque -> Bool) -> b -> b)
  -> ((Ataque -> Dir) -> b -> b -> b)
  -> Red -> b
foldRed t s fw sw (Terminal n)      = t n
foldRed t s fw sw (Servidor n r)    = s n (foldRed t s fw sw r)
foldRed t s fw sw (Firewall f r)    = fw f (foldRed t s fw sw r)
foldRed t s fw sw (Switch fsw ri rd) = sw fsw (foldRed t s fw sw ri) (foldRed t s fw sw rd)

recRed
  :: (Potencia -> b)
  -> (Potencia -> Red -> b -> b)
  -> ((Ataque -> Bool) -> Red -> b -> b)
  -> ((Ataque -> Dir) -> Red -> b -> Red -> b -> b)
  -> Red -> b
recRed t s fw sw (Terminal n)      = t n
recRed t s fw sw (Servidor n r)    = s n r (recRed t s fw sw r)
recRed t s fw sw (Firewall f r)    = fw f r (recRed t s fw sw r)
recRed t s fw sw (Switch fsw ri rd) = sw fsw ri (recRed t s fw sw ri) rd (recRed t s fw sw rd)

```

Punto 4 — versiones con fold

```

combinarFirewalls' = foldRed Terminal Servidor combinar Switch

```

```

intensidadRestante' a r = foldRed
    (\n a -> Just (a - n))
    (\n rs a -> rs (a + n))
    (\f rs a -> if f a then rs a else Nothing)
    (\fsw rsi rsd a ->
        case fsw a of
            Der -> case rsd a of
                (Just aIzq) -> rsi aIzq
            _ -> case rsi a of
                (Just aDer) -> rsd aDer
    ) r a

pasos' :: Red -> Ataque -> (Int, Maybe Ataque)
pasos' = foldRed
    (\n a -> (1, Just (a - n)))
    (\n rs a -> addFst (+1) (rs (a+n)))
    (\f rs a -> if f a then addFst (+1) (rs a) else (0, Nothing))
    (\fsw rsi rsd a ->
        case fsw a of
            Der -> case addFst (+1) (rsd a) of
                (pasosDer, (Just ataqueRes)) -> addFst (+pasosDer) (rsi ataqueRes)
            _ -> case addFst (+1) (rsi a) of
                (pasosIzq, (Just ataqueRes)) -> addFst (+pasosIzq) (rsd ataqueRes)
    )

rutasAServ' :: Ataque -> Red -> ([Ruta], Maybe Ataque)
rutasAServ' a r = foldRed
    (\n a -> ([], Just (a - n)))
    (\n rs a -> addRutasFst ([] :) (addRutasFst (map (Forward :)) (rs (a+n))))
    (\f rs a -> if f a then addRutasFst (map (Forward :)) (rs a) else ([], Nothing))
    (\fsw rsi rsd a ->
        case fsw a of
            Der -> case addRutasFst (map (Der :)) (rsd a) of
                (rs, Just ataqueRes) ->
                    addRutasFst ((rs++) . (map (Izq :))) (rsi ataqueRes)
            _ -> case addRutasFst (map (Izq :)) (rsi a) of
                (rs, Just ataqueRes) ->
                    addRutasFst ((rs++) . (map (Der :))) (rsd ataqueRes)
    ) r a

```